

10_llmsverifier_helixagent

LLMsVerifier × HelixLLM / HelixAgent — Verification Extension Design (T1)

Date: 2026-07-06 **Scope:** Extend LLMsVerifier to verify local (HelixLLM / llama.cpp) + new cloud providers flawlessly, source CONST-040 capability flags from REAL probes (never static literals), and meet the CONST-037 24h / CONST-038 60s freshness contracts. **Method:** Read of actual code under submodules / llms_verifier/llm-verifier and submodules/helix_agent/internal (structs pasted verbatim, no guessing per §11.4.6) + cited web research on llama.cpp / Ollama / OpenAI-compatible verification best practice.

1. Current State — ACTUAL structures (pasted from source)

1.1 Capability model — capabilities/types.go

ProviderCapabilities carries a rich, verification-fillable capability set. The two CONST-040-relevant members:

```
// capabilities/types.go
type ProviderCapabilities struct {
    Provider    string
    Model       string
    Verified    bool
    VerifiedAt  time.Time
    // ...
    Protocols  []ProtocolType // MCP / ACP / LSP / gRPC /
                OpenAI / Anthropic / Ollama
    Model_     ModelCapability `json:"model_capabilities"`
    Extended   ExtendedCapabilities
    Custom     map[string]interface{}
}

type ModelCapability struct {
    Vision, ImageInput, ImageOutput, Audio, Video, PDF, OCR bool
    FunctionCalling, ToolUse, Embeddings, CodeExecution,
        WebBrowsing, Reasoning bool
}
```

```

    MaxContextTokens, MaxOutputTokens int
}

const ( // ProtocolType
    ProtocolMCP    = "mcp"
    ProtocolACP    = "acp"
    ProtocolLSP    = "lsp"
    ProtocolOllama = "ollama" // etc.
)

```

Finding: MCP/ACP/LSP are modelled as Protocols []ProtocolType, NOT as dedicated per-model booleans, and there is **no RAG / Skills / Plugins member at all**. CONST-040 requires MCP, LSP, ACP, Embedding, RAG, Skills, Plugins — only Embedding (ModelCapability.Embeddings) and MCP/LSP/ACP (as protocol tags) exist today.

1.2 Per-model DB record — database/database.go (already CONST-040-shaped)

```

// database/database.go
type VerificationResult struct {
    ID, ModelID int64
    VerificationType string
    StartedAt time.Time; CompletedAt *time.Time
    Status string; ErrorMessage *string
    ModelExists, Responsive, Overloaded *bool
    LatencyMs *int
    SupportsToolUse, SupportsFunctionCalling bool
    SupportsCodeGeneration, SupportsCodeCompletion,
        SupportsCodeReview bool
    SupportsEmbeddings, SupportsReranking bool
    SupportsImageGeneration, SupportsAudioGeneration,
        SupportsVideoGeneration bool
    SupportsMCPs, SupportsLSPs, SupportsACPs
        bool // <-- CONST-040 booleans EXIST here
    SupportsMultimodal, SupportsStreaming, SupportsJSONMode,
        SupportsStructuredOutput bool
    SupportsReasoning, SupportsParallelToolUse bool
    // ... CodeLanguageSupport []string, CodeDebugging,
        TestGeneration, etc.
}

```

Finding: The DB layer already stores per-model SupportsMCPs/LSPs/ACPs/Embeddings/... booleans + freshness columns (StartedAt, CompletedAt, Status). It has **no RAG / Skills / Plugins column**. This struct is the natural home

for CONST-040 flags — the gap is population + a few missing columns, not architecture.

1.3 In-memory verification result — llmverifier/models.go

```
type VerificationResult struct {
    ModelInfo ModelInfo; Availability AvailabilityResult;
    ResponseTime ResponseTimeResult
    FeatureDetection FeatureDetectionResult; CodeCapabilities
    CodeCapabilityResult
    GenerativeCapabilities GenerativeCapabilityResult;
    PerformanceScores PerformanceScore
    Timestamp time.Time; Error string; ScoreDetails ScoreDetails
}
type Capabilities struct { // ModelInfo.Capabilities
    Completion, Chat, Embedding, FineTuning, ImageGeneration,
    CodeGeneration,
    ToolUse, Multimodal, FunctionCalling, Voice, Rerank bool
}
```

1.4 Verification / probe flow (what is REAL vs static today)

Component	File	Behaviour
Detector.DetectProviderCapabilities	capabilities/ detector.go	Starts from static GetProviderBaseCapabilities (registry), then runs REAL o (SSE content-type probe) + c (GET /v1/models). Sets Ver. VerifiedAt=now, 15-min T Endpoint maps are cloud-o anthropic/deepseek/gemini mistral). MCP/LSP/ACP are they come from the static r
capabilities/registry.go	—	25+ providers with hardco []ProtocolType{...Prot LSP}. This is the CONST-040 a capability flag set from a
llmverifier/verifier.go Verify()	—	The REAL engine. detectF testEmbeddings (real POS embeddings), testMultim per-language coding probes LLMClient(endpoint, ap
	—	

Component	File	Behaviour
llmverifier/llm_client.go NewLLMClient(endpoint, apiKey, headers)		Plain endpoint string → a l host:port/v1 works out- every OpenAI-shaped prob
verification/verification.go Verify() (VerificationService)	—	Returns honest sentinel ErrVerificationNotWire true bluff was removed) — llmverifier.Verifier.
parallel-tool-use probe	verifier.go	Returns honest (false, 0 has no ToolCalls field , so counting is impossible toda

Net current-state verdict: the *real-probe engine exists and is endpoint-agnostic*; the *DB schema for capability booleans exists*. The gaps are (a) MCP/LSP/RAG/Skills/Plugins are not probed (static or absent), (b) local providers are absent from the endpoint/registry maps, (c) tool-calling verification is blocked by a missing ToolCalls field, (d) VerificationService.Verify is unwired, (e) no 24h/60s freshness policy.

1.5 HelixAgent consumption side

- internal/llm/provider.go — LLMProvider interface exposes GetCapabilities() *models.ProviderCapabilities.
 - internal/models/types.go ProviderCapabilities — has SupportsStreaming/FunctionCalling/Vision/Tools/Search/Reasoning/CodeCompletion/CodeAnalysis/Refactoring but **NO MCP/LSP/ACP/RAG/Skills/Plugins booleans** → CONST-040 gap on the consumer struct.
 - internal/llm/providers/helixllm/provider.go — **ALREADY EXISTS**. Endpoints: chat /v1/chat/completions, embeddings /v1/embeddings, models /v1/models, health /internal/health, default https://localhost:8443. Implements HealthCheck, Complete, GetCapabilities. (Sibling lmstudio, ollama providers also present.)
 - internal/verifier/ (discovery.go, service.go, scoring.go, health.go, startup.go, subscription_detector.go, adapters/) — the consumption package.
 - internal/services/provider_discovery.go — defines LLMsVerifierScoreProvider interface (GetProviderScore, GetModelScore, RefreshScores), useDynamicScoring=true default, and ProviderDiscovery{Verified bool; VerifiedAt time.Time; Capabilities *models.ProviderCapabilities}. **The freshness plumbing (VerifiedAt + RefreshScores) already exists** — it needs a 24h/60s policy bound to it.
-

2. Extension Design

2.1 CONST-040 capability model — add the missing dimensions (types + DB)

A. **capabilities/types.go** — extend **ModelCapability** with the CONST-040 dimensions currently missing, each a probe-set boolean (never a literal):

```
type ModelCapability struct {
    // ...existing...
    // CONST-040 capability integration (all set by real probes,
    // §2.4)
    MCP      bool `json:"mcp"`           // tool-calling over MCP-
        shaped tool spec
    LSP      bool `json:"lsp"`           // code-intelligence contract
    ACP      bool `json:"acp"`           // agent-communication
        protocol
    RAG      bool `json:"rag"`           // retrieval-augmented / long-
        context grounding
    Skills   bool `json:"skills"`       // skill/plugin dispatch
    Plugins  bool `json:"plugins"`
    // provenance – anti-bluff §11.4.69
    Evidence map[string]CapabilityEvidence
        `json:"evidence,omitempty"`
}
```

```
type CapabilityEvidence struct {
    Probe      string `json:"probe"`           // e.g.
        "tool_calls_array_present"
    Endpoint   string `json:"endpoint"`       // real URL hit
    Observed   string `json:"observed"`       // captured wire
        snippet (redacted)
    ArtifactPath string `json:"artifact_path"` // §11.4.69
        evidence file
    At         time.Time `json:"at"`
    OK         bool      `json:"ok"`
}
```

B. **database/database.go VerificationResult** — add SupportsRAG bool, SupportsSkills bool, SupportsPlugins bool columns (+ migration) so the DB single-source-of-truth carries all seven CONST-040 flags. SupportsMCPs/LSPs/ACPs/Embeddings already exist.

C. **Retire registry-static protocol flags as *authoritative*.**

GetProviderBaseCapabilities stays as a *default/seed* only; after a live

verification run the probe-derived `ModelCapability`.

`{MCP,LSP,ACP,RAG,Skills,Plugins}` + Evidence **override** it, and only probe-derived values are exported to `HelixAgent`. A capability with no `CapabilityEvidence` **MUST** serialise as `false` (fail-closed) — this is the mechanical anti-bluff guarantee (CONST-040 / §11.4.69).

2.2 Local HelixLLM / llama.cpp verification path

HelixLLM speaks the OpenAI-compatible surface (llama.cpp `llama-server`: `/health`, `/props`, `/v1/models`, `/v1/chat/completions`, `/v1/embeddings`, `/slots`, `/metrics` [1][2]). Because `LLMClient` already takes a raw endpoint, verification reduces to **registration + probe orchestration**, not new transport code.

New local provider descriptor (config-driven, no hardcoded host — §CONST-045-style): register a `local/helixllm` provider whose endpoint = HelixLLM base URL (e.g. `http://127.0.0.1:PORT`), `apiKey` optional. Add it to `capabilities/detector.go` `getProviderModelsEndpoint` / `getProviderStreamEndpoint` maps **derived from config**, not literals.

Verification sequence for a local endpoint:

1. **Health** — GET `/health`; ready iff `200 {"status":"ok"}` (llama.cpp returns `503` while the model loads [1][2]). For HelixLLM's own `/internal/health` use that path. Sets `Responsive`, `LatencyMs`.
2. **Model list** — GET `/v1/models`; each `data[].id` → a models row. `ModelExists=true`. (llama.cpp returns the single loaded model; HelixLLM may return several.)
3. **Server props (local-only enrichment)** — GET `/props`; parse `modalities.vision` (→ vision candidate) and `embedding/pooling` config; llama.cpp tool-calling requires the server started with `--jinja` [2] — capture that as a precondition, but still *probe* it (do not trust props alone).
4. **Real completion probe** — POST `/v1/chat/completions` minimal prompt; `non-empty choices[0].message.content ⇒ Chat/Completion=true`, records latency.
5. **Tool-calling probe** — §2.4.
6. **Embedding probe** — §2.4.
7. **Vision probe** — §2.4.
8. Compose `VerificationResult`, persist, set `CompletedAt`, `Status=completed`.

2.3 New-provider registration (stream-06 providers + local)

A provider becomes verifiable by supplying a **ProviderDescriptor** (config/registry row), not code:

```

type ProviderDescriptor struct {
    ID          string          // "helixllm", "local",
               "<new-cloud>"
    BaseURL     string          // from config/.env – never a literal
               (§CONST-045)
    AuthType    AuthType      // api_key | bearer | none
    APIKeyEnv   string          // e.g. HELIXLLM_API_KEY
               (never the key itself, §11.4.10)
    Wire        string          // "openai" | "anthropic" |
               "ollama"
    HealthPath  string          // "/health" | "/internal/
               health"
    ModelsPath  string          // "/v1/models" | "/api/tags"
    ProbeMatrix []CapabilityProbe // which §2.4 probes to run
}

```

Registration = append a descriptor (from config) → Detector/
 llmverifier.Verifier pick it up. Wire="ollama" selects the /api/tags + /
 api/show discovery path (Ollama now exposes a capabilities array:
 completion, tools, insert, vision, embedding, thinking [3][4]) — but
 capabilities are still confirmed by live probes, never trusted from the metadata
 field.

2.4 Anti-bluff capability probes (each flag = one real probe + captured evidence)

Every CONST-040 flag is set by exactly one probe that hits a real endpoint and captures a wire artefact to docs/qa/<run-id>/ (§11.4.69). No probe hit ⇒ flag stays false.

Flag	Probe	Real request	PASS c (captu
Embeddings	embedding probe	POST /v1/embeddings {input:"probe",model}	HTTP 2 data[a non- of len ≥ needs]
FunctionCalling/ ToolUse/MCP	tool-call probe	POST /v1/chat/completions with a tools: [{type:function,...get_weather}] + a prompt that forces a call (tool_choice:"required" where supported)	respon messag with funct. JSON a pseudo

Flag	Probe	Real request	PASS c (captu
			(Requi ToolCa Messag llama.o jinja
Reasoning	reasoning probe	chat request with a reasoning prompt / reasoning_effort	non-en thinkin correct answer
Vision	vision probe	POST /v1/chat/completions with an image_url/base64 content part (a tiny known fixture image)	correct the fixt (metan fixture correct §11.4.1 modals a <i>preco</i> proof
RAG	long-context grounding probe	inject a unique fact in a long context, ask for it back	verbat injecte against PASS)
ACP	TestACPs (exists)	agent-protocol handshake probe	protoco respon
LSP	code-intelligence probe	code-completion/analysis request	structu comple
Skills/Plugins	skill-dispatch probe	request naming a registered skill/plugin	model the cor
Streaming	detectStreaming (exists)	streaming request	Conten event chunk

Prerequisite fix (blocker): add `ToolCalls []ToolCall` to `llm_client.go` Message so the tool-call/MCP probe can count real calls (removes the honest `(false, 0)` sentinel). This is the single highest-leverage code change.

Analyzer self-validation (§11.4.107(10)): each probe's classifier is paired with a golden-good and golden-bad fixture in unit tests — a probe that PASSes its golden-bad recording is itself a bluff gate and fails meta-test.

2.5 Freshness — CONST-037 (24h) + CONST-038 (60s)

The plumbing exists (VerifiedAt on both the verifier ProviderCapabilities and HelixAgent ProviderDiscovery; RefreshScores). Bind a policy:

- **Producer (LLMsVerifier):** replace the flat 15-min detector cache with a **staleness policy**: a VerificationResult older than **24h** (now - CompletedAt > 24h) is marked stale and re-queued for a background re-verification run. This is the CONST-037 “verified within 24h” guarantee at the source.
- **Consumer (HelixAgent):** ProviderDiscovery must **hide/annotate** any model whose backing VerificationResult.CompletedAt is > 24h old (CONST-037: display only models verified within 24h).
- **Status propagation ≤ 60s (CONST-038):** expose a lightweight status delta — either (a) a push channel (the internal/messaging/event_stream.go seam already imports the verifier) or (b) a **poll of the verifier status endpoint at ≤ 60s** in provider_discovery’s refresh loop, updating VerifiedAt/Status. Whichever is available, the interval **MUST** be ≤ 60s so a model going unhealthy/unverified reflects in HelixAgent within a minute.
- **Anti-bluff:** freshness timestamps come from real CompletedAt of a real probe run — never time.Now() stamped without a run (that would be the classic metadata-only bluff, §11.4/§11.4.5).

2.6 Wiring VerificationService.Verify

Resolve req.ModelID → models PK → dispatch llmverifier.Verifier probes (§2.4) against the model’s provider endpoint → compose + persist a database.VerificationResult with all seven CONST-040 flags + CapabilityEvidence. This removes ErrVerificationNotWired and makes the single-source-of-truth entripoint real.

3. Top risks

1. **Tool-calling verification is blocked** until Message.ToolCalls is added; without it MCP/FunctionCalling flags cannot be truthfully set (they must stay false — CONST-040 fail-closed). Highest-priority code change.
2. **Static-registry protocol flags** (registry.go) are a live CONST-040 anti-bluff hazard: any consumer reading Protocols today gets literal-sourced MCP/ACP/LSP, not probed. Must be demoted to seed-only and overridden by evidence-backed probes, or the “single source of truth” is lying.
3. **Consumer struct gap:** HelixAgent models.ProviderCapabilities has no MCP/LSP/ACP/RAG/Skills/Plugins fields, so even a correct verifier cannot deliver CONST-040 flags end-to-end until the consumer struct +

provider_discovery mapping are extended and the $\leq 60s/24h$ policy is enforced.

Sources verified 2026-07-06

- [1] llama.cpp server endpoints (health/models/props/embeddings/slots/metrics), DeepWiki: <https://deepwiki.com/ggml-org/llama.cpp/6.2-llama-server-http-api>
- [2] llama.cpp tools/server/README.md — /health 503-until-ready, /props modalities.vision, tool calling requires --jinja, /v1/embeddings pooling=none, mmpoj for vision: <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>
- [3] Ollama capabilities via /api/tags + /api/show (completion/tools/insert/vision/embedding/thinking), Issue #5794: <https://github.com/ollama/ollama/issues/5794>
- [4] Ollama /api/tags docs + capability enhancement PR #10174: <https://docs.ollama.com/api/tags>, <https://github.com/ollama/ollama/pull/10174>
- [5] LM Studio tool-use verification over /v1/chat/completions + /v1/models check: <https://lmstudio.ai/docs/developer/openai-compatible/tools>
- [6] Native message.tool_calls vs pseudo-text (OpenAI-compatible tool_choice=required), lmstudio-bug-tracker #2115: <https://github.com/lmstudio-ai/lmstudio-bug-tracker/issues/2115>
- [7] LLM tool-calling accuracy tester (precision/recall/param-accuracy) — probe design precedent: <https://github.com/adamwlarson/LLMToolCallingTester>
- [8] vLLM tool-calling feature reference: https://docs.vllm.ai/en/latest/features/tool_calling/

Code read (facts, no guessing — §11.4.6): submodules/llms_verifier/llm-verifier/{capabilities/types.go, capabilities/detector.go, capabilities/registry.go, database/database.go, llmverifier/models.go, llmverifier/verifier.go, llmverifier/llm_client.go, verification/verification.go}; submodules/helix_agent/internal/{llm/provider.go, llm/providers/helixllm/provider.go, models/types.go, services/provider_discovery.go, verifier/*}.